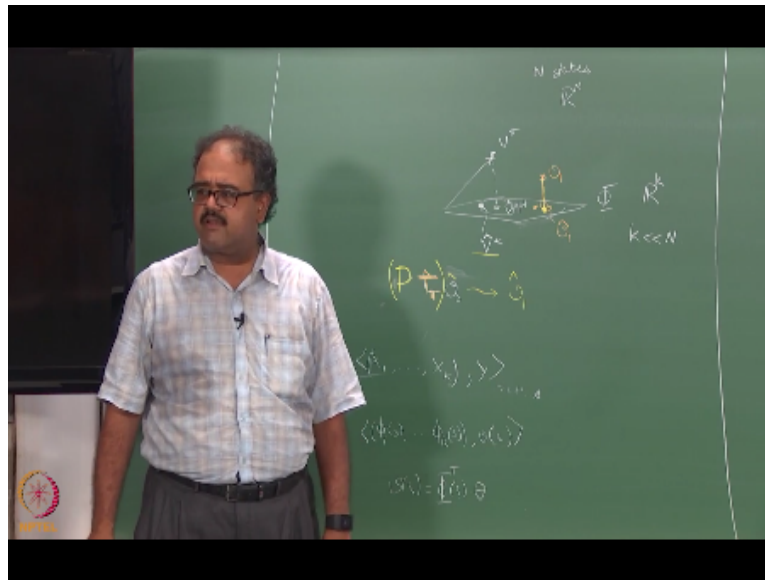


NPTEL
NPTEL ONLINE COURSES
REINFORCEMENT LEARNING
LSTD AND LSTDQ

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

so what exactly does linear function approximation do, right so ideally what you would want it to do is the following right so, so I have some abstract space in which I have some $v \pi$ that is being represented, right, so I have an abstract space in which $v \pi$ is represented and there is some plane right, there is some space which is which kind of is spanned by my feature vectors ϕ , right there is some space that is spanned by my feature vectors ϕ right but there is my real value function $v \pi$ sitting in some other space this is clear, right.

(Refer Slide Time: 01:11)



So what space would be $v \pi$ reside in, suppose I have N states right, so $v \pi$ resides in a R^N state space right so it is an R^N state space, where each coordinate corresponds to the value of the respective state, right so the first coordinate will be the value of the first state the second

coordinate will be the value of the second state and so on so forth, it will be a point in $\mathbb{R}^N$, right and my ϕ might be something that spans a much smaller state space right my ϕ could be something like $\mathbb{R}^k$ where, ideally, I would like that to be the case because I do not want to be representing an N dimensional state in which case I can represent the tabular form right I can represent the actual value function right.

So I do not want this k should be far far less than N , so in this case I am assuming that N is 3 and k is 2 okay just so, that I can draw any other higher dimensions it will be hard for me to draw, right so ideally what would you want to happen, right let us say that, right so that is ideally that is what I would want my $\hat{v}^\pi$ to be right, that is the closest possible I can get to given that I have to stay in this plane, right people must have done some kind of linear regression curve fitting and things like, projections right this is essentially what you are doing is the closest projection to the plane, right.

So this is essentially what we are trying to look for right and what exactly is this in terms of, regression fits that you are getting, the least squares solution to the fit, right so the least square solution to the regression problem that you have, where your targets are drawn from v^π , right will essentially be the $\hat{v}^\pi$, right it turns out that there are some results that show that if you are actually going to be updating v^π , $\hat{v}^\pi$ incrementally right.

So essentially I have this vector θ which I will be updating incrementally under certain conditions on ϕ and so on so forth, right your $\hat{v}$ will not converge to this so we will call this $\hat{v}^*$ star, right so I am not going to go into the proof of that thing if you are interested I can give you pointers, good when I talked about pointers Varun just walked in you can put the Psitsiklis, and van Roy paper on the convergence of TD with value function approximation, right.

Maybe you can give a link to the paper on, I think I used to link that paper any way from the previous editions but since the previous edition seem to have disappeared you can add the link, to the paper right, so $\hat{v}^*$ would be the least squares projection but it turns out you will not get it normally so what you will end up doing, what you can end up showing is you will converge to

some $\hat{v}$ right which is within a certain distance of $\hat{v}^*$ right that distance to which you converge is bounded right.

So that so it is within a bounded distance of $\hat{v}^*$ you will converge okay so this is basically what you can show, so I so let us not call that $\hat{v}^*$ I am going to call that $\hat{v}$ optimal right that is the actual optimal thing I want and $\hat{v}^*$ is the one that I will get at, convergence right so my $\hat{v}^*$ will be very close to $\hat{v}$ optimal but except in some rare cases it will not be $\hat{v}$ optimal okay, so why you think this is happening ideally the best solution because anyway you remember we are minimizing the least squares error right we are minimizing least squares right that is a way we wrote the error function right.

So ideally the minimizer should be $\hat{v}$ opt right why are we not converging to $\hat{v}$ opt, sample error.. noise.. what kind of noise, nonstationarity. so that is the problem right not just when you say sample error it is essentially the target keeps changing over time right I pointed this out last class also, right the target that you are updating towards keeps changing over time because you are using the $\hat{v}$ itself, as your target that keeps changing over time so that introduces a little complication in the dynamics.

So you cannot guarantee that you will converge to $\hat{v}$ opt, right if you knew the true v π values right if you knew the true v π values you will converge to $\hat{v}$ opt, since you do not know the true v π values and you are essentially boot strapping you end up converging somewhere $\hat{v}^*$ okay, the nice thing is at least that given some mild conditions on ϕ , the distance between $\hat{v}$ opt and $\hat{v}^*$ is bounded, okay.

So that is the nice thing, that is what makes, what mild conditions they had to be orthogonal the ϕ s have to be orthogonal, and then you have a linear, not necessarily orthogonal yeah linear independence is fine they have to be linearly independent and you have to sample on policy, right so on policy sampling is very important, if you are linear function approximation

with dynamic programming style updates can be shown to diverge, so dynamic programming does uniform sweeps right it is not on policy, every state is of, every state is updated equally often, right so that is certainly not on policy and you can show that that will actually diverge right.

So think of what is possibly happening in the in the update set up right so I start off with some, suppose I suppose I want to do an updation on my let us just say even on this case right I have a fixed π right so what do I start off with, I start off with some guess for v , say I would start off with some v not right so I have, start off with some representation for v not, right so that will be somewhere in this space, then what do I do, some kind of an update operation right so this is I am just using my, bellman operator to summarize it but if it is Q learning or if it is TD learning for everything I can write a corresponding operator right.

So the TD learning operator would essentially be some kind of a sampled version of this bellman operator right so essentially I apply some kind of an approximation of the bellman operator to my v and what will I do I will get a v_1 , right but my v_1 will typically be not here, so I start off with some v not right there is no approximation here right I start off with a v not, I apply $T \pi$ on it right whether it is a one round of TD update right or anything else I apply something on it, right then what will essentially happen is this will go to v_1 , v_1 right, so v not when I apply π on it, $T \pi$ on it, it will go to v_1 right what do I do at that point I project it back because that is all I can represent right.

So that is the best I can do in some sense what I am essentially doing is projecting it back till it hits the space so that is my v_1 hat, so I apply a, see normally the projection matrix, projection operator is indicated by P or Rho right so I am using P here even though P we are using for transition probability I am also using P for the projection right so I am do a projection so in some sense, that is a joint operation I use to go from v_0 hat to v_1 hat right so in this case there is no approximation really because I started there but I can do this so this takes me to v_1 hat right and I keep doing this again and again until I converge to, do I do it that way if you think of the update that I am doing right I just change the θ s directly right, the way I change the θ s is I find the

gradient of the performance in the direction, find, then change the θ in the opposite direction of the gradient right.

So that is essentially what I do, I just do gradient descent, I look at the approximation error right and I go in the opposite direction, so where did I compute the new v_1 , I am not, I am not explicitly computing v_1 right I basically computed $\hat{v}_1$ anyway conceptually what we are doing is this kind of a, well you can think of it as a composition right I applied T followed by P , I mean if the word joint is confusing you okay it is a composition operator, right so this is essentially what is happening in terms of what we are trying to do in function approximation right.

So one thing which people started thinking about is, hey we are trying to solve a regression problem right, can we use can you set it up as a proper regression problem, right so what do I mean by set it up as a proper regression problem how do you solve how do you normally solve linear regression problems you are given some data set you are given some x_1 to x_k as your input dimensions and then you are given some y right.

so like that you are given many many such data vectors, and then you form a matrix out of it so you form A , what is what is the matrix called what is the matrix called, data matrix yeah you should know a different name for it, you have sat through enough of Nandan's classes, no no, that is different, anyone else from DoMS here, no one right, you are the only DoMS representative I am going to pick on you, it is sometimes called the design matrix, right so the data that you get is also called the design, I am pretty sure many of you have heard that term, design matrix, it essentially means that that data set X , all the X s

Right so the capital X that you write by combining all the X s is called design matrix, so normal regression problem will require us to have something like, so I will have an input vector and a Y and then I will have this indexed over right if I want to do something like this for, value function approximation what should I do I should have some vector S so basically we have right.

So this is what I should have right this should be the data that I should feed to the regressor, like this I should have some N samples makes sense right the only problem here is my v of s , I do not know my v of s , so what we do for v of s right I am going to assume that my v of s comes from right, some ϕ transpose s into θ right you could if you want to solve this as a Monte carlo, if you want to do TD then you use this, you can set it up as a Monte carlo problem and then you could just take the return starting from that state and then give that as the sample of the $v \pi$, you could set up a function like that, that is a perfectly valid way of doing this right but uninteresting, right, why is it uninteresting, because what you will converge to we talked about this right, even the, I had this question in the exam also right.

So will you converge to if you use Monte Carlo, a fixed sample Monte Carlo is the least squares minimizer of the prediction of the return right but what you will converge to if you use TD for a fixed sample is the certainty equivalence estimate right the Markov model, that is a more interesting quantity to estimate than just the least squares estimator, right, anyway, right so we can assume that $v s$ is given by this right and for my current setting of θ right I could just plug this in here right.

So whatever is my, I would make an initial guess for θ then I make one big data set like this then what I do I choose my favorite least squares solver right and get a new estimate, for θ right so then I keep doing this again and again right so now if you think about it my input is in the form of some ϕ s right and the output is in the form of some ϕ times θ right now I can actually try to solve this in some kind of a, sorry, what are all the data points here, you are talking about all the states we will come to the data generation part little later, almost surely you will probably not have samples for all the states but then for some state you might have more samples right, great, so now let us let us look at it in a, so what I really want is suppose I have this this ,whole operation that is happening right my P times $T \pi$ of v , of $v \pi$ hat equal to $v \pi$ hat right this is my fixed point right this is this is essentially what I want to achieve right I want to get to my fixed point solution.

So if I get to my fixed point what will I have so what will this look like, now I fixed my π right so this will become my $P \pi$ if you remember the matrix $P \pi$ that we had earlier so that is

essentially the transition reduced to what happens when I take my action corresponding to the state, so instead of $P(s, a, s')$, right, it will be $P(s, \pi(s), s')$ right so that is this matrix so I have reduced it, right I can remove the action column from there so this is essentially $P(s, s')$ right where the action is fixed to be π of s we looked at this matrix when we saw the convergence and the of the bellman operator right now what is remaining I really have to look at the projection business, right.

So I have written the T_π operator here, I now have to look at what the projection is going to look like, what will the projection look like what is X , what is X here, right that is the projection right you remember the linear regression solution okay where ϕ is written for all these states, right ϕ of s is one row corresponding to state when I do not write any arguments to ϕ it is the entire matrix so each row corresponds to one state right so I have written the entire bellman equation there for me, right so it turns out that with this with a slight change because I will use it later, I will just write it down here, I will use it later I can also write this for, where this one will be taken appropriately for s' right.

So whatever is π of s' , so q of q, π of s' , $\pi s'$ okay, that will be the q, π here and one other change is ϕ will be ϕ of s, a , right ϕ will be defined on state action pairs, right remember we talked about how would you go from v, π to q, π I gave you several methods right one of them was to have $\phi s, a$, right instead of ϕs , right I am assuming people remember anything we did in the last class right, sorry I am assuming people remember everything we did in the last class, right and so that is the change here right, so ϕ will be $\phi s, a$ right, and this is this is this is different and this you will have to be careful about okay.

So otherwise you can write something very similar to what we did for v okay, so now what do we do, now that we have this, I can write this as, that is the entire vector okay so this is a ϕ matrix, right $\phi \theta$ so each row will get multiplied by θ so that will give me the whole vector v , right this is v, π , yeah so perils of on the fly notation translation whenever I make whenever I put some symbol there which you have not seen before please stop and ask me it might be something which I forgot to translate anyway right.

(Refer Slide Time: 24:07)

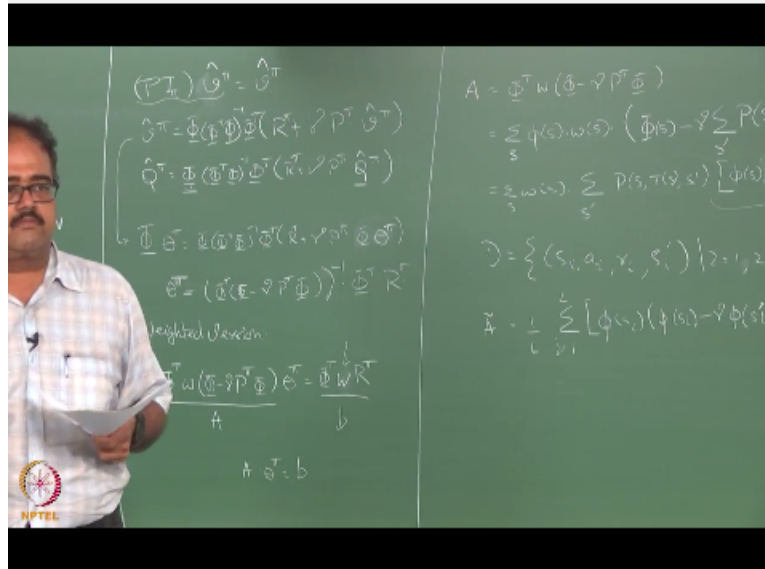
So now I am going to do a little bit of the thing, so one of the so this is this is this gives you the least square solution yeah sorry, R , R should be R^{-1} , yes, everywhere right, so this is the least square solution right so what is what is striking about this least square solution is that it treats everything as an equal contributor towards the error right, every state as an equal contributor towards error right every sample as an equal contributor towards the error but then we know that that is not always the right thing to do, why, well we have a small enough expression power right.

So we really want to approximate those states that are more important for us right and then leave out those states that are not that important, right so what you do essentially is instead of looking at minimizing a least squares fit you minimize a weighted least squares fit so then what would happen is in your solution you will have a weight cropping up, at different points, right so what we will do is we will rewrite this and while I do the rewriting I will also introduce the weight, right I am not going to redo the whole thing again so I am going to write a weighted version, where w is some kind of a weight function over all states, right that tells you how much you want to weigh a state by, what is the typical weight function that you want to use, the steady-state probability distribution right steady state probability distribution it could be anything else, whatever takes your fancy right.

So you could use that weighted probability distribution so this is essentially what we get so I am going to make it little simple I am going to call that A , I am going to call this b , so I end up getting A into, so the rewriting is not just for fancy okay, so this is going to allow us to do little look at each one of those quantities there little closely, and see what they are actually doing so A and b , okay, yes but now we are not doing any incremental updates or anything right, I am giving you a bunch of samples and I am solving for it, yeah yeah I want to approximate the steady state distribution okay and if I do the sampling on policy I will approximate it, that is what I was missing, yeah.

So that is a good point, so all I am talking about now is getting the least squares fit for a given set of samples, given a fixed set of samples I am trying to get a least squares fit right and I can get that least squares fit regardless of what that w is I will get a least squares fit given a set of samples, right I have no guarantee about how good that fit will be from the viewpoint of my true $\hat{v}^{\text{opt}}$, right if you use any arbitrary thing, in terms of how close I am to $\hat{v}^{\text{opt}}$, I have no no guarantees if I use any arbitrary w right.

(Refer Slide Time: 40:24)



So we know that if it is on policy, right then we will get that $\hat{v}^*$ given infinite number of samples I mean of course so as it is all asymptotic convergence guarantees right the more samples you are given the closer you will be to be $\hat{v}^*$ right but you will never get to $\hat{v}^*$ opt right and that is guaranteed if your w is going to be your on policy distribution right but if I am looking at any fixed set of samples and I am talking about just getting some estimate that fits that sample, I can choose whatever w I want, right I said you would want it to be the steady state distribution but even if you choose something else.

It is fine why would you want to choose something else, could it be because you do not know the steady state distribution right you do not know the steady state distribution one way of approximating it is just to look at the relative frequencies in the samples that you have drawn already, right that could be very very far away from the steady state distribution, infact we end up using that distribution only because you do not know what the steady state distribution is so you end up using the distribution that you infer from the samples that you have drawn, right, are people with me or people lost, no okay see I said that I mean if w should be the steady state distribution probability I said and all of you agreed to that, and that is the ideal situation that we want right.

But we do not know what the steady state distribution is I mean you would not know that until you either you solve the Markov chain set up by the policy and then you know what it is but for solving that you need the transition probabilities, so that is not going to happen so you do not know what the true steady state distributions are, right how do you approximate the steady state distributions by counting it from the sample that you have right, right when you count from any fixed set of samples right

The distribution that you infer can be something very very far away from the true w that you want right the weights that you get by inferring from a fixed set of data could be very far from the w that you want, if the data is very very large right as t tends to infinity doing an incremental update will approximate the w , but we are not talking about t tending to infinity we are talking about a fixed set of samples, here so given a fixed set of samples the w that you get can be very far away from the true w so you just live with it okay.

So let us go and look, write out a right, now this is essentially my $P \pi$ that transpose comes from, this whole thing gets modified because I went from a matrix notation to a individual entry notation right so essentially if you look at what is the products that are being computed here, this will work out correctly okay so I flipped the, if you think, do not think I have flipped the transpose that is because I went from the matrix to component wise notation right.

So this ϕs times ϕs is the product that you have here, right and it will sum over all s' , so this will drop out okay so essentially you are getting your $\phi s \phi s$ product from here, right essentially I have added an additional unnecessary $\phi s a s'$ to that product so that I can take it inside the, inside the summation, okay does it make sense right so I have for this this term, I have added in unnecessary $P s s'$ term to it.

So that I can take it inside the inner summation right so that is essentially what I have done the rest of it is the same, right nothing else no no fancy tricks here right the reason we did this is so that we can write this as some kind of a sampled thing this kind of an expectation so if you think about it, what is it, it is some kind of a weighted sum of this quantity, right so what I can do now I can keep sampling transitions from s to s' according to this distribution right compute this

quantity, the difference between s and s' right I compute this quantity again and again by sampling according to this distribution, right and then take a average weighted by how much importance I want to give to each of the states, and that gives me my, a right.

So we got around one of the biggest problems nobody actually asked me how can is how can I say I have solved the linear regression problem here, when I actually have the $P \pi$ in here, right I have the $P \pi$ in here and I have R in here right I cannot say that I have solved the linear regression problem from samples, right because I am assuming that I know the model and if know the model I might as well solve the model directly right why am I doing this the whole idea of doing all of this is I do not know the model, right.

So I need to somehow I figured out what my $P \pi$ is and what my $R \pi$ is instead of figuring out P and R directly right so we can have a actually a way of estimating the A matrix directly, right so what I can say is given a set of samples, right of the form my samples are going to be of the form I am going to get L such samples of this form okay remember, so this is the current state action reward and the next state okay that is that is the form of samples I am going to get ok.

So in our normal notation it will be S_t at $t+1$ S_{t+1} right, the i index just denotes this is the i th sample ok nothing to do with the time indexing, right does that make sense right this i this i has nothing to do with our time index it just denotes it is the i th sample okay so I am going to get states like this now what I can do is I can form an estimate for A tilde, this is my $\tilde{A}$, okay so A is fine what about b , sorry, I am not a hundred percent sure yeah p is just a number right so the whole vector is the ϕ ok so it is fine right.

So what about b , can you do b also something like this, we can write, it is almost almost clear, what is it, good question yeah how does where did the w go, yeah this will get, I am kind of wrapping the w and the P into the frequency right so yeah we are making a very strong assumption that everything is fine, right so that the number of times S occurs here right is exactly the number of times you want it according to the w right then it is fine, ok so I can do something ok let me do b first before I move on to anything else, right

So that is what $R \pi$ is, people remember the $R \pi$ thing right $R\pi$ is essentially summation over all s' , $P s \pi s'$ into $R s \pi s'$, so which is essentially the expected reward that you are going to get I mean we wrote it as an expectation I am just writing it as a simple thing here, ok and I can do the same trick that I did last time, I took the ϕ inside, right so just like last time I have taken the ϕ inside so what I can do now is same thing that we did earlier for A so I can write b tilde as, sorry, right I can do that so I can also write the action version of this right.

So yeah, here, there is no transpose, so like I was saying right the ϕ transpose here, you lose the transpose when I write it component wise, so I can write the action version of it also like this right remember we wrote the same thing earlier here, so I can run through this simplify the whole thing right and I can get the same thing here right except I am leaving out this part I want you to fill it in tell me what should go there, ok here is the tricky part I might not have this in my data, in S_i' I might not have done, $\phi S_i'$, so we have to be careful about whenever we use this things right.

So you make sure that you have done πS , that is the on policy part of it right and likewise I can do the b tilde here this is a relatively straightforward in fact that is what I wrote first, so this kind of an approach where we use an estimated A tilde and b tilde, right and solve for a value function π is called LSTD, least squares TD, right so when you operate with the v function it is LSTD when you operate with the Q function it is, close, it is called, LSTDQ.

So you have LSTD and LSTDQ and you also have an LSTD λ , where you do this for TD λ as well, ok there things become little tricky, so here in some sense my A is a repository of my model right so in the LSTD λ , the gamma λ terms will appear here also, here the gamma term appears, right when you do LSTD λ you will get a gamma λ term in your A matrix right.

So so that is the trickier part right so that is essentially what will happen in, the difference between LSTD and LSTD λ , it is very convenient, because it looks like it is a very small change in fact right and is there something else that you note here so when I writing my solution for θ I am solving for θ right, in case you forgot with all this mechanizations we are doing we are actually solving for $\theta \pi$ right.

So when you solve it for θ π I have A at this side and b that side right I am going to do now $\tilde{A} \tilde{b}$ and what is the one thing that you notice if I have to $\tilde{A} \tilde{b}$, it should be invertible let us assume that if ϕ s are, if my ϕ s are linearly independent it will be invertible right I can show little linear algebra I can show that if the ϕ s are linearly independent, okay this will be linearly, this will be invertible and then, what else my, one by L will go away right.

So the one by L, I do not have to carry it around I can just throw it away because it will get cancelled out on the A and b, right one by L will go away then what are you left with, just a summation of terms of this side right, so what does this tell you that as I keep getting more transitions, I can just keep adding to the A matrix it is very very easy to update my A matrix incrementally right I can just keep doing this iterations right and then as I am sampling more states I just keep adding to the A matrix so I can get a much larger, I can keep improving my estimate of the value function right as I keep getting more transitions so at any point of time I can just take my current state of the A matrix and solve it for v π or q π , okay great so this is called LSTD or LSTDQ.

IIT Madras Production
Funded by
Department of Higher Education
Ministry of Human Resources Development
Government of India
www.nptel.ac.in